Quantum Information Processing (2026) 25:49
https://doi.org/10.1007/s11128-025-05032-w

Hybrid quantum–classical machine translation

Mina Abbaszadeh[1,2] · Mariam Zomorodi[3] · Mehrnoosh Sadrzadeh[1] · Vahid Salari[4] · Philip Kurian[2]

Received: 1 March 2025 / Accepted: 30 November 2025 / Published online: 3 February 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

This study investigates the feasibility of language translation using quantum natural language processing (QNLP) on noisy intermediate-scale quantum devices. Classical NLP struggles with large-scale computations, but QNLP leverages quantum properties to process linguistic data more efficiently, potentially advancing NLP applications. We propose a hybrid quantum–classical model for neural machine translation, which may outperform classical methods whenever practical quantum computers are available. Using the Shannon entropy, we demonstrate the importance of rotation gate angles in parameterised quantum circuits, enabling communication between circuits for different languages. We combine bag-of-words and compositional structure models, presenting the first proof-of-concept for compositional QNLP, where circuit parameters and structure are key to model interpretability. Focusing on sentences with similar structures, we implement an encoder–decoder model with long short-term memory (LSTM) networks for translation. Experiments on 160 English–Persian sentence pairs achieved the best results with Adam as the optimiser on a two-layer LSTM, minimising loss relative to both stochastic gradient descent and root-mean-square propagation (RMSprop).

✉ Mariam Zomorodi
zomorodi@pk.edu.pl

✉ Mehrnoosh Sadrzadeh
m.sadrzadeh@ucl.ac.uk

✉ Vahid Salari
vahid.salari1@ucalgary.ca

Philip Kurian
pkurian@howard.edu

1 Department of Computer Science, University College London, London, UK

2 Quantum Biology Laboratory, Howard University, Washington DC, USA

3 Department of Computer Science, Cracow University of Technology, Kraków, Poland

4 Institute for Quantum Science and Technology, Department of Physics and Astronomy, University of Calgary, Alberta, Canada

Keywords Quantum natural language processing · Machine translation · Shannon entropy

1 Introduction

Natural language processing (NLP) refers to the use of computers for processing spoken and written language, including analysing and understanding speech or writing produced in the format and structure of natural language, as well as producing it. Alan Turing proposed NLP in 1950 as a criterion for automated interpretation and generation of natural language, known today as the Turing test [1]. Notably, in the context of the Turing test for CUBBITT,[1] most participants struggle to distinguish CUBBITT translations from human translations [2, 3]. In recent years, significant progress has been made in NLP with the development of advanced language models such as the Generative Pre-trained Transformer 3 (GPT-3) and its massive 175 billion parameters [4] and ChatGPT with many more parameters [5]. This represents a major milestone in the field of NLP, enabling the performance of tasks like machine translation, sentiment analysis, and text summarisation with a level of fluency that was once thought impossible—paving the way for the development of sophisticated language models like ChatGPT that have revolutionised artificial intelligence (AI). As NLP continues to evolve, we will see even more impressive language models emerge in the coming years.

Moreover, the recent progress in quantum computation and information has opened up new opportunities for NLP and other fields with broad applications [6–22]. Beyond language understanding, quantum artificial intelligence (QAI) is growing in different areas such as image processing, pattern recognition, and data compression [11, 23]. Recent studies have demonstrated that quantum machine learning algorithms can enhance feature extraction and improve image quality through quantum-enhanced optimisation and entanglement-based representations [23–25]. These developments highlight the versatility of quantum circuits as trainable models, supporting our motivation to apply similar architectures to natural language processing tasks. Quantum natural language processing (QNLP) aims to define the necessary processes and mappings for NLP tasks on quantum computing devices. Specifically, new quantum approaches for NLP have been developed with the potential to outperform classical counterparts in the future [26, 27]. One potential advantage of QNLP is its ability to process large amounts of data simultaneously, which could be useful in training language models. Quantum computers can also perform certain operations faster than classical computers, which could potentially speed up the training process and lead to more accurate models. Another area where QNLP could be useful is in the analysis of context and meaning in language. Quantum systems are inherently probabilistic, which could allow for a more nuanced understanding of language and a better ability to capture the subtle nuances of meaning and context (Fig. 1).

In the QNLP model, language is interpreted as quantum processes through the use of a diagrammatic formalism in categorical quantum mechanics [28]. QNLP protocols

[1] Charles University Block-Backtranslation-Improved Transformer Translation.

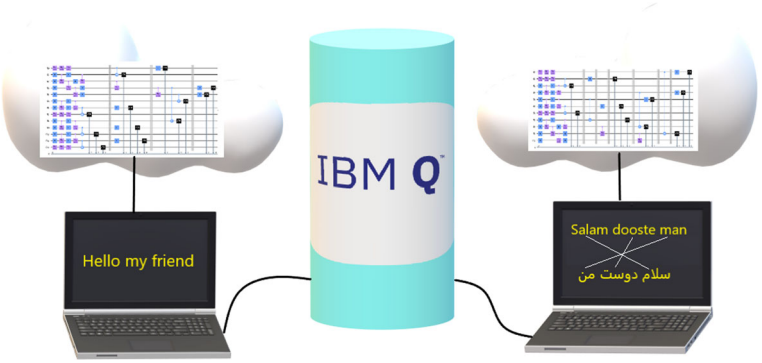


Fig. 1 Quantum machine translation on noisy intermediate-scale quantum (NISQ) devices. Any syntactically distinct languages can be considered in this approach. Here is an example of a symbolic representation of translation between English and Persian on an IBM quantum computer

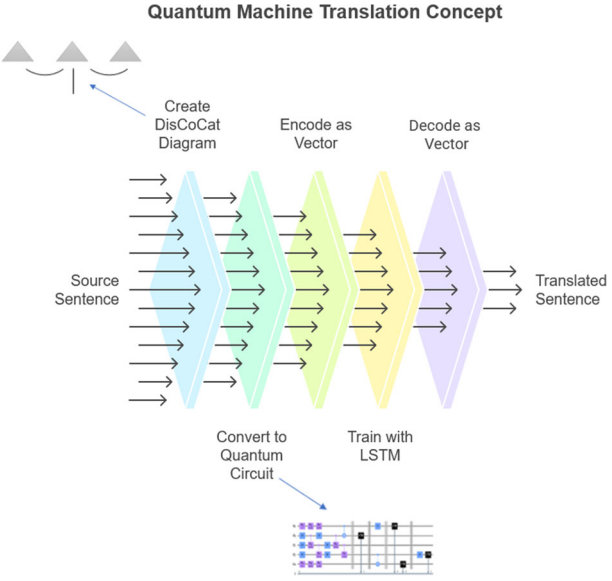


Fig. 2 This figure demonstrates different steps of the algorithm for the translation task via QNLP

have two aspects: syntax and semantics, which are performed through mathematical operations (Fig. 2). Semantics for quantum protocols are provided by compact closed categories [29]. Semantics in QNLP are demonstrated by encoding the information flow between words within a sentence. The compact closed category is used to encode this information flow. As displayed in Fig. 3, cups and lines present the information flow between words of sentences.

The Distributional Compositional Categorical (DisCoCat) model of QNLP, introduced by Coecke et al. [30], utilises a compositional grammar model, specifically the pregroup grammar, to introduce the structure of sentences. The information flow and encoded correlations between all words in a sentence are embedded into a vec-

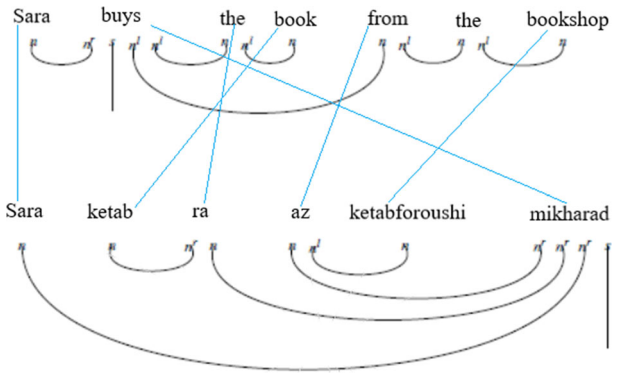


Fig. 3 Top and bottom diagrams represent DisCoCat diagrams of English and Persian, "Sara buys the book from the bookshop" and "Sara ketab ra az ketabforoushi mikharad," respectively

tor space, where vector geometry captures the correlations between words within a given sentence. The methods for parsing sentences via pregroup grammar and drawing DisCoCat diagrams are described in various articles [30, 31].

The ZX-calculus https://zxcalculus.com/ is a graphical language that extends beyond circuit diagrams, enabling the generalisation of Pauli-Z and Pauli-X operations to demonstrate properties of circuits, entanglement states, and protocols in a visually succinct yet logically complete manner. The ZX-calculus is a powerful tool that is shaping the next generation of quantum software. By utilising the calculus, optimisation strategies can be employed that perform state-of-the-art T-count reduction, a key metric for fault-tolerant computing, and gate compilation. The generators of the calculus correspond closely to the basic operations of lattice surgery in the surface code, providing a visual design and verification language for these codes.

Quantum-circuit optimisation techniques based on the rules of the ZX-calculus have shown promise in simplifying ZX-diagrams [32]. However, this approach works best when there are few non-Clifford gates in the original circuit. By utilising different quantum circuit optimisation techniques, the performance of quantum machine learning (QML) circuits can be improved and their cost can be reduced.

In Ref. [33], the use of the ZX-calculus for converting DisCoCat diagrams of sentences into quantum circuits is discussed in detail. For more information on this topic, please refer to Refs. [26–34]. Additionally, Dimitri Kartsaklis et al. have developed a high-level Python library for QNLP called Lambeq [35], which implements all stages of a pipeline for converting sentences into string diagrams, tensor networks, and quantum circuits ready for use on a quantum computer.

In Ref. [36], an algorithm was presented for translating QNLP tasks using long short-term memory (LSTM). Recurrent neural networks (RNNs) are commonly used in machine learning for sequence-to-sequence tasks such as machine translation [37] and speech synthesis. RNNs can be considered as multiple copies of the same network, where each network passes a message to its successor. LSTM is a special type of RNN that can learn long-term dependencies.

In Ref. [38], the systems are trained using similarity scores in basic NLP rather than deep learning models. The primary difference between deep learning and other machine learning approaches lies in the way they learn and represent data. In traditional machine learning, such as logistic regression or decision trees, the algorithm is provided with a set of features engineered by humans, and it then learns to associate those features with the target output. Deep learning, on the other hand, uses a neural network architecture that is capable of automatically learning hierarchical representations of data. These neural networks are composed of many layers of interconnected nodes, and each layer learns to represent a more complex feature of the data. The deeper layers of the network learn to represent more abstract and high-level features, while the earlier layers learn to represent simpler features. This ability to learn hierarchical representations of data has led to deep learning models achieving state-of-the-art performance in many machine learning tasks, including NLP, computer vision, and speech recognition. In summary, the key difference between deep learning and other machine learning approaches lies in the way they learn and represent data. Deep learning models utilise a hierarchical neural network architecture to learn complex representations of data, yielding superior performance in various machine learning tasks.

Several approaches to the concept of quantum machine translation are presented in the academic literature. For instance, one study proposes the implementation of a quantum neural network-based machine translator specifically designed for English language translations [39]. Another scholarly article serves as an introductory guide to the field of quantum machine learning, which has significant implications for applications such as image and speech recognition or strategic optimisation [12, 40]. Another approach outlines the development of a deep learning system that is optimised for machine translation [2]. Additionally, a systematic review of existing literature pertaining to quantum machine learning is available [41, 42]. Lastly, QNLP has been developed to represent the meaning of phrases as vectors encoded in quantum computers.

In our work, an LSTM model that has shown good performance on non-trivial tasks such as sequence learning was constructed. To feed the model with inputs, the quantum circuit was encoded into a multidimensional space of vectors that corresponded to the embedding of the initial words. This encoding was used to train an LSTM model, which was later decoded to the corresponding quantum circuits.

The paper is structured as follows: Section 2 provides a summary of all the methods used in the paper. Section 3 describes the DisCoCat diagram and how it is converted into a quantum circuit. In this section, a dataset of sentences from two different languages is provided, and the Shannon entropies of their quantum circuits are evaluated. Section 4 presents an implementation for the QNLP translation task using a classical LSTM model. In this section, we describe how a dataset of 160 sentences in English and Persian was prepared, and how the parameterised quantum circuit of similar sentences in English and Persian was designed.

2 Methods

We utilised DisCoCat diagrams to represent sentences from two syntactically distinct languages (English and Persian). Through the application of the specified operations, these diagrams were transformed into quantum circuits. A dataset of quantum circuits was then compiled, containing both Persian and English sentence circuits. The English sentence circuits were separated into two groups, with one group having identical angles of rotation gates for synonymous words in English and Persian sentences. The angles of rotation gates were then altered to create another group of English quantum circuits, where synonymous words between English and Persian sentences did not share the same angles for their rotation gates. Subsequently, the quantum circuits were simulated for 20,000 shots on the IBMQ-qasm-simulator, and the resulting frequency distributions were used to generate probability distributions. Specifically, after running a quantum circuit, the occurrence frequency of each state is divided by the sum of all occurrence frequencies to calculate the probability of that state.

We propose a novel method for the purpose of aligning source and target sentences. Our method involves running quantum circuits on both the source and target sentences, and subsequently evaluating the Shannon entropy from the output probability distributions of the resulting circuits. We then measure the difference between the Shannon entropy values of each pair of source and target sentences. Our method has yielded an almost regular pattern of differences, which is expected to guide quantum neural machine translation in avoiding the training of obviously incorrect sentences in explicit direct mapping models. Moving forward, we intend to add a new layer to recurrent quantum neural networks using this approach, which can be applied to various datasets to ensure their synchronisation. This approach has also been implemented in classical neural networks [43]. Moreover, we created a dataset consisting of 160 quantum circuits for English and Persian sentences, with synonymous words from both languages having identical parameters. By utilising classical LSTM, we were able to perform the translation task via the encoding-decoding of quantum circuits of sentences. In Fig. 2, we present all steps for our goal.

3 Quantum circuits of sentences in two different languages and evaluating Shannon entropy

The aim of QNLP is to create a framework that captures both semantic and syntactic aspects of sentences using a mathematical form known as the compact closed category. Quantum maps are introduced to represent meaning in QNLP and developed using a diagrammatic language for representing processes and their compositions [44, 45]. This non-commutative categorical quantum logic language allows for the reduction of diagrams of sentences and facilitates the comparison of grammatical structures in different languages. Pregroups are utilised to encode the grammar of languages, providing an algebraic analysis of sentence parsing [46]. Models for the semantics of positive and negative transitive sentences and relative pronouns are presented in Refs. [30, 31].

3.1 Conversion of sentences to quantum circuits

In this section, we will summarise the conversion of a sentence into a quantum circuit using a mathematical framework, with complete explanations in Ref. [36]. Vector spaces and pregroups are used to assign meanings to words and grammatical structures to sentences in a language, which can then be converted into quantum circuits. A compact closed category is a monoidal category where for each object A there are objects A^r and A^l, and morphisms

$$\eta^l : I \to A \otimes A^l, \quad \eta^r : I \to A^r \otimes A$$
$$\epsilon^l : A^l \otimes A \to I, \quad \epsilon^r : A \otimes A^r \to I.$$

In the graphical language, the η maps are depicted by caps, and the ϵ maps are depicted by cups [30]. A pregroup is a partially ordered monoid $(P, \leqslant, \cdot, 1)$ with multiplication monoid '$\cdot$' and unit '1'. For each $p \in P$ there are elements p^l, $p^r \in P$ such that

$$p^l \cdot p \leqslant 1 \leqslant p \cdot p^l, \quad p \cdot p^r \leqslant 1 \leqslant p^r \cdot p$$

Pregroup is a compact closed category. Morphisms are reductions and the operation '$\cdot$' is the monoidal tensor of the monoidal category. In order to transition from classical to quantum computing in QNLP, each sentence is encoded as a quantum circuit to be executed on a quantum computer. In this process, we follow the steps outlined below:

Initially, we select synonymous sentences in English and Persian. As an example, we consider the English sentence "Sara buys the book from the bookshop" and its Persian counterpart "Sara ketab ra az ketabforoushi mikharad". Next, we utilise pregroup grammar to analyse and parse the sentences [30]. Let the basic types be:

n : noun,
s : declarative statement

and generate the free pregroup of this type [36]. If the types of words in a sentence are reduced to the basic type "s" through juxtaposition, the sentence is considered grammatical.

It is important to note that the relationship $n^l.n \leqslant 1$ holds, which implies the existence of a morphism $\epsilon^l : n^l \otimes n \to I$ within a compact closed category, with symbols representing the various components.

The assignment of types to words within a sentence is determined by their grammatical position. In the example "Sara buys the book from the bookshop", we connect all words of the sentence to the verb "buys". Let's consider the words "the" and "bookshop" as an example. In this case, we need to establish a connection between the word "bookshop" and "the". To represent the noun "bookshop" we assign the type "n", and to connect it to the word "the", we use n^l. Both "the" and "bookshop" should also be connected to the word "from". Consequently, we assign "n" to the word "the" and adjoin it to "from" using n^l. To connect "Sara" to the verb, we put the right-adjoint n^r to "buys". Similarly, by assigning "n" to the word "from", we adjoin it to the verb "buys" using n^l. This process continues for all words in the sentence. These assign-

ments are not unique for words, and they change according to the structure of the sentences. For more comprehensive information, please refer to [47]. According to [47], the example sentence “Sara buys the book from the bookshop" has the following type of assignment:

$$\begin{array}{cccccccc} \text{Sara} & \text{buys} & \text{the} & \text{book} & \text{from} & \text{the} & \text{bookshop.} \\ n & (n^r s n^l n^l) & (nn^l) & n & (nn^l) & (nn^l) & n \end{array}$$

This is grammatical because of the following reduction:

$$(nn^r)sn^l(n^l n)(n^l n)n(n^l n)(n^l n) \to s(n^l n) \to s.$$

According to [46], the example sentence “Sara ketab ra az ketabforoushi mikharad" has the following type of assignment:

$$\begin{array}{cccccc} \text{Sara} & \text{ketab} & \text{ra} & \text{az} & \text{ketabforoushi} & \text{mikharad.} \\ n & n & (n^r n) & (nn^l) & n & (n^r n^r n^r s) \end{array}$$

This is grammatical because of the following reduction:

$$n(nn^r)nn(n^l n)n^r n^r n^r s \to nn(nn^r)n^r n^r s \to n(nn^r)n^r s \to (nn^r)s \to s.$$

The reductions and types are interpreted as linear maps and vector spaces, obtained by a monoidal functor from a pregroup to finite-dimensional vector spaces. For the above sentences, the reductions are depicted diagrammatically in Fig. 3, where each cup has two wires, i.e. it has two inputs. For more details, see [36].

Now, by using the ZX-calculus, we translate DisCoCat diagrams into quantum circuits. We consider one qubit for every wire of type n and s. According to [30], wires are the input and output of morphisms. According to the definition of a pregroup, n^l and n^r are of type n, named left and right adjoints of n. So the verb is a state of some qubits. Figure 3 shows that the quantum circuit for the English sentence consists of 13 qubits, 1 for “Sara”, 4 for “buys”, 2 for “the”, 1 for “book”, 2 for “from”, and 1 for “bookshop”. The quantum circuit for the Persian sentence consists of 11 qubits, 1 for “Sara”, 1 for “ketab", 2 for “ra”, 2 for “az”, 1 for “ketabforoushi”, and 4 for “mikharad”. This comes directly from the fact we have assigned 1 qubit to noun wires and 1 qubit to sentence wires. DisCoCat diagrams of sentences are constructed based on reduction maps and the representation of ϵ maps using cups. In these diagrams, the number of connections to each word corresponds to the number of wires present.

Each noun qubit corresponds to two R_x rotations and one R_z rotation, which together implement an Euler decomposition to determine the qubit’s state. The four verb qubits must be connected, which is achieved using instantaneous quantum polynomial (IQP) layers consisting of Hadamard gates and diagonal unitaries. In this specific circuit, a single IQP layer is used, where all qubits receive Hadamard gates and the 1st and 2nd, 2nd and 3rd, and 3rd and 4th qubits are connected by a conditional R_z gate (a diagonal unitary), resulting in all four qubits being linked to represent the state of the English verb “buys”. This method is applied similarly to Persian sentences. The theory behind IQP circuits and why they are a good choice for QML is explained in detail in the paper [17]. The cups (tensor contractions in standard DisCoCat) correspond to Bell effects and require post-selection where we measure all qubits many times and keep

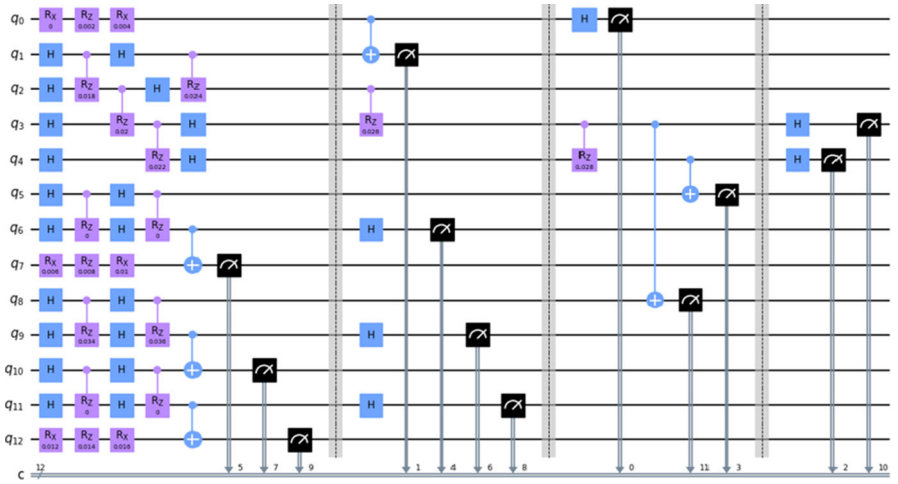


Fig. 4 The quantum circuit of "Sara buys the book from the bookshop" in English

the shots where 12 out of the 13 qubits (excluding the sentence qubit) are zero. It is possible to have more than one IQP layer.

In Qiskit format, Figs. 4 and 5 represent the quantum circuit of the sentence "Sara buys the book from the bookshop" in English and Persian, respectively. We identify the same angles for rotation and controlled rotation gates for synonymous words in each pair of quantum circuits for English and Persian sentences. We select synonymous keywords that are the same between the English and Persian sentences and set the same angles for their rotation gates. Other words can take random angles for their rotation gates. Because two syntactically distinct languages will not have a one-to-one mapping between corresponding words, future work will entail how multiple-word syntactic constructions in one language map to various multiple-word alternatives in another. The structure of complex phrases in English [31] and the rules of more complex parsing protocols [35] could be used to distinguish assignment patterns for basic types and morphisms between noun-centric (thing-oriented) and verb-centric (process-oriented) languages.

3.2 Dataset creation for evaluating Shannon entropy

Shannon entropy is a widely used measure that quantifies the level of statistical uncertainty or randomness in a system. Derived from the Boltzmann entropy in the microcanonical ensemble, it has proven to be particularly useful in assessing the amount of information present in a given signal or dataset. In the context of machine learning algorithms, Shannon entropy is a useful tool to measure the dissimilarity or distance between different samples or clusters of data. Alternatively, Shannon entropy can be used to measure distance by computing the entropy of each sample or cluster and comparing it to the overall dataset's entropy. However, care should be taken: Consider the case where there is only one state in each distribution. The probability

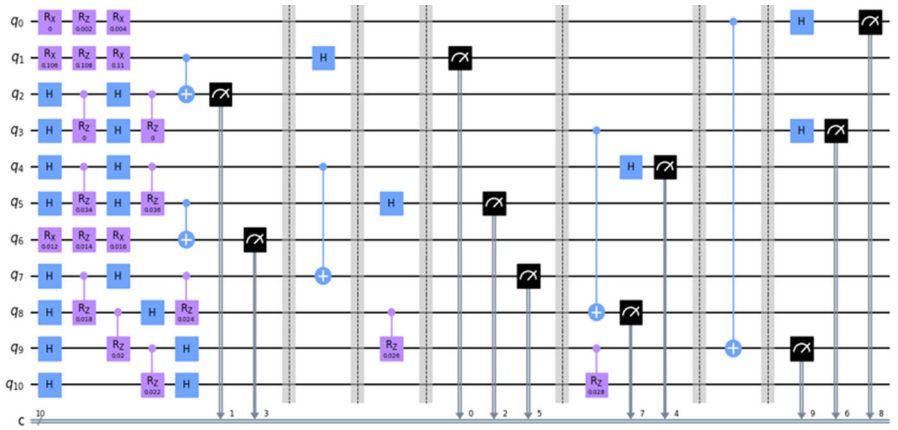


Fig. 5 The quantum circuit of “Sara ketab ra az ketabforoushi mikharad" in Persian

of that state is one, and so the Shannon entropy is zero for each distribution. However, two such states could be separated by a very large distance (e.g. orthogonal “0” and “1” signals). A much “closer” state to either distribution could be constructed, which has more Shannon entropy but is much closer to the signal in the one-state distribution with zero entropy. This should be addressed with care for each use case and identified as a possible source of error in analysis.

Furthermore, Shannon entropy can also be applied to feature selection and dimensionality reduction in machine learning. The entropy of each feature can be calculated, and the feature with the highest information can be chosen as the best representation of the dataset. It is important to recognise that while Shannon entropy is a valuable metric for measuring differences in machine learning algorithms, its effectiveness is contingent upon the unique characteristics of the dataset and the problem at hand. Although our study focuses on the English and Persian languages, it is essential to acknowledge that other languages may also be relevant. Table 1 displays several example sentences from the aforementioned languages, and each sentence was run through a quantum circuit and evaluated using Shannon entropy on the IBMQ-qasm-simulator with 20,000 shots. After running a quantum circuit that contains n qubits, we get 2^n different states and their frequencies. The frequency of each state represents the number of occurrences of that state after 20,000 shots. By dividing the frequency of each state x_i by 20000, we get the probability of that state as $p(x_i)$. Let X represent the probability distribution of running the quantum circuit for a given sentence. The Shannon entropy H is calculated as follows:

$$H(X) = -\sum_{i \in I} p(x_i) \log p(x_i)$$

For each state $x_i \in X$, $p(x_i)$ is the probability of x_i. I is the index set that represents the 2^n elements of X. This means that I represents the number of states observed in the circuit execution results file for a sentence.

In this research, we put the same angles for synonymous words in quantum circuits of English and Persian sentences and evaluate the Shannon entropy of each quantum circuit. Next, we proceed to swap the angles of rotation gates of each quantum circuit for the English sentence randomly and repeat the experiment, ensuring that the angles corresponding to synonymous words in the English and Persian sentences are not the same. This is done in order to maintain the original angles of each circuit as much as possible. In other words, the angles of rotation gates of a quantum circuit for each English sentence are swapped, and the experiment is repeated. An illustrative example of this process can be seen in Fig. 6. The new angle of each rotation gate is selected randomly from the original angles. For example, for the two sentences—'Sara buys the book from the bookshop' in English and 'Sara ketab ra az ketabforoushi mikharad' in Persian, the synonymous words are:
Sara ≡ Sara
buys ≡ mikharad
book ≡ ketab
bookshop ≡ ketabforoushi
from ≡ az
For the above synonymous words, we first set the same angles for their rotation gates, as presented in Figs. 4 and 5. Then, we swapped angles in each English circuit and repeated the experiment.

3.3 The results for evaluating Shannon entropy of quantum circuits of sentences

The Shannon entropy metric may prove to be a valuable tool in analysing QNLP data. In this section, we have conducted a thorough analysis of the quantum circuits of sentences, which involved two primary steps. We conducted experiments using quantum circuits to compare synonymous sentences in English and in Persian. First, to ensure consistency, in each quantum circuit, we used similar angles of rotation gates for words with similar meanings in each sentence. Second, we adjusted the angles of rotation gates in quantum circuits of English to ensure that synonymous words in both languages did not share the same angles. Thus, we sought a way to match each pair of English and Persian sentences and add a matching layer in the recurrent neural network. The resulting circuits were executed using the IBMQ-qasm-simulator with 20,000 shots, and the corresponding Shannon entropy values for each circuit are presented in Table 2. The third column of Table 2 depicts the Shannon entropy values for English sentences with angles exchanged, while Table 3 shows the discrepancies between the first and second, as well as the second and third columns of Table 2. The results of this analysis are presented in Fig. 7. Second, we utilised the left and middle columns of Table 2, which assigned similar angles of rotation gates to the synonymous words in quantum circuits of English and Persian sentences, and calculated the Shannon entropy of each quantum circuit of all English and Persian sentences listed in Table 1. The results of this analysis are presented in Fig. 8.

For the results that are illustrated in Fig. 7, the blue line represents the differences in Shannon entropy between English and Persian circuits with the same angles of rotation gates for synonymous words, while the orange line represents the discrepancies for

Table 1 The table below presents synonymous sentences in two Source and target languages. For each sentence in the source language (left column), its translation appears directly in the right column

Source language (English)	Target language(Persian)
a.Sara buys the book from the bookshop	a′.Sara ketab ra az ketabforoushi mikharad
b.John buys the book from the bookshop	b′.John ketab ra az ketabforoushi mikharad
c.Sara buys the notebook from the bookshop	c′.Sara daftar ra az ketabforoushi mikharad
d.Sara walks in the park	d′.Sara dar park ghadam mizanad
e.Bob walks in the park	e′.Bob dar park ghadam mizanad
f.John walks in the park	f′.John dar park ghadam mizanad
g.Sara and Alice walk in the park	g′.Sara va Alice dar park ghadam mizanand
h.Sara and Susan walk in the park	h′.Sara va Susan dar park ghadam mizanand
i.John and Bob walk in the park	i′.John va Bob dar park ghadam mizanand
j.Bob and Alice walk in the park	j′.Bob va Alice dar park ghadam mizanand
k.Sara sees Bob	k′.Sara Bob ra mibinad
l.John sees Alice	l′.John Alice ra mibinad
m.Bob sees Alice	m′.Bob Alice ra mibinad
n.Bob sees John	n′.Bob John ra mibinad
o.Bob sees Sara	o′.Bob Sara ra mibinad
p.Bob sees Susan	p′.Bob Susan ra mibinad
q.John sees Bob	q′.John Bob ra mibinad
r.Sara sees Alice	r′.Sara Alice ra mibinad
s.Sara sees John	s′.Sara John ra mibinad
t.Sara sees Susan	t′.Sara susan ra mibinad

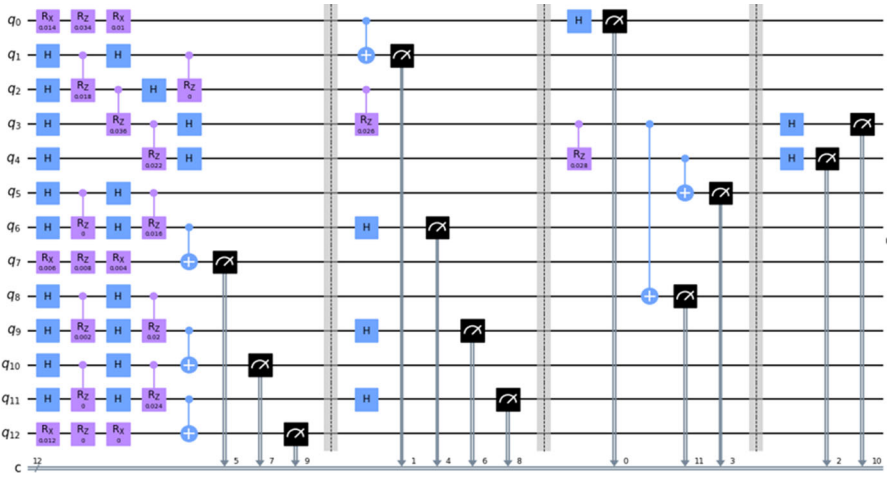


Fig. 6 The quantum circuit of "Sara buys the book from the bookshop" after swapping angles of rotation gates of Fig. 4. We maintain all gates and the overall structure of the circuit

Table 2 The results of evaluating the Shannon entropy of quantum circuits for source (English) and target (Persian) sentences are shown in the left and middle columns, respectively. For $x = a, \ldots, t$, $H(x)$ and $H(x')$ are the entropies of the source sentence and its translation, respectively. In both cases, synonymous words in the sentences were assigned the same angles in quantum circuits. The right column displays the results of evaluating the Shannon entropy of quantum circuits for English sentences after angle swapping. $H(x'')$ shows the entropy of the source sentence after swapping angles

Shannon entropy of quantum circuits of English sentences	Shannon entropy of quantum circuits of Persian sentences	Shannon entropy of quantum circuits of English sentences
$H(a) = 6.001315160873559$	$H(a') = 5.0031967710951015$	$H(a'') = 6.004736428636913$
$H(b) = 6.006578931000881$	$H(b') = 5.006429642328229$	$H(b'') = 6.018957659399867$
$H(c) = 6.005966787750403$	$H(c') = 5.005974227809354$	$H(c'') = 6.017595798609497$
$H(d) = 6.995594756756778$	$H(d') = 5.997804218054402$	$H(d'') = 6.996647874624478$
$H(e) = 6.995539523537989$	$H(e') = 5.99550229119629$	$H(e'') = 7.04742881564586$
$H(f) = 6.995713266260664$	$H(f') = 5.998383911173536$	$H(f'') = 7.005377911621739$
$H(g) = 7.472076459921503$	$H(g') = 6.419834213204461$	$H(g'') = 7.259668533783197$
$H(h) = 7.58181670756182$	$H(h') = 6.59739502901938$	$H(h'') = 7.352212286404398$
$H(i) = 7.315157675950335$	$H(i') = 6.335068026209578$	$H(i'') = 7.171110796325161$
$H(j) = 7.6592276186465345$	$H(j') = 6.640685720752816$	$H(j'') = 7.3294011238656$
$H(k) = 2.0006784957219317$	$H(k') = 2.999789504246745$	$H(k'') = 2.0049399527709584$
$H(l) = 2.5203329918706316$	$H(l') = 3.4914554204229953$	$H(l'') = 2.510412409090681$
$H(m) = 2.6645482358797676$	$H(m') = 3.6707868709904896$	$H(m'') = 2.1451431789662303$
$H(n) = 2.330285405108207$	$H(n') = 3.3187826378940786$	$H(n'') = 2.2337291491135707$
$H(o) = 2.2449753288648266$	$H(o') = 3.2293084705194266$	$H(o'') = 2.067748117020039$
$H(p) = 2.8533918863400323$	$H(p') = 3.828552220833362$	$H(p'') = 2.259740814655625$
$H(q) = 2.4035414027599233$	$H(q') = 3.332105656985817$	$H(q'') = 2.131743480610954$
$H(r) = 2.4224007881842406$	$H(r') = 3.430680111906539$	$H(r'') = 2.19789321159273$
$H(s) = 2.0807634799644323$	$H(s') = 3.08513013099599$	$H(s'') = 2.0414209912678674$
$H(t) = 2.602246530030625$	$H(t') = 3.611661842800734$	$H(t'') = 2.437846467686803$

Table 3 The left column shows discrepancies between the left and the middle columns of Table 2. The right column shows discrepancies between the middle and the right columns of Table 2

Discrepancies between entropies	Discrepancies between entropies
$\lvert H(a) - H(a')\rvert = 0.998118389778458$	$\lvert H(a'') - H(a')\rvert = 1.001539657541812$
$\lvert H(b) - H(b')\rvert = 1.000149288672652$	$\lvert H(b'') - H(b')\rvert = 1.012528017071638$
$\lvert H(c) - H(c')\rvert = 0.999992559941049$	$\lvert H(c'') - H(c')\rvert = 1.011621570800143$
$\lvert H(d) - H(d')\rvert = 0.997790538702376$	$\lvert H(d'') - H(d')\rvert = 0.998843656570076$
$\lvert H(e) - H(e')\rvert = 1.00003723234167$	$\lvert H(e'') - H(e')\rvert = 1.05192652444957$
$\lvert H(f) - H(f')\rvert = 0.997329355087128$	$\lvert H(f'') - H(f')\rvert = 1.00699400044820$
$\lvert H(g) - H(g')\rvert = 1.052242246717042$	$\lvert H(g'') - H(g')\rvert = 0.839834320578736$
$\lvert H(h) - H(h')\rvert = 0.98442167854244$	$\lvert H(h'') - H(h')\rvert = 0.75481725738508$
$\lvert H(i) - H(i')\rvert = 0.980089649740757$	$\lvert H(i'') - H(i')\rvert = 0.836042770115583$
$\lvert H(j) - H(j')\rvert = 1.018541897893724$	$\lvert H(j'') - H(j')\rvert = 0.329826494780934$
$\lvert H(k) - H(k')\rvert = 0.9911100852481$	$\lvert H(k'') - H(k')\rvert = 0.994849551475787$
$\lvert H(l) - H(l')\rvert = 0.971122428552364$	$\lvert H(l'') - H(l')\rvert = 0.98104311332314$
$\lvert H(m) - H(m')\rvert = 1.006238635110722$	$\lvert H(m'') - H(m')\rvert = 1.52564369202425$
$\lvert H(n) - H(n')\rvert = 0.988497232785871$	$\lvert H(n'') - H(n')\rvert = 1.085053488780508$
$\lvert H(o) - H(o')\rvert = 0.9843331416546$	$\lvert H(o'') - H(o')\rvert = 1.1611560353499396$
$\lvert H(p) - H(p')\rvert = 0.97516033449333$	$\lvert H(p'') - H(p')\rvert = 1.568811406177742$
$\lvert H(q) - H(q')\rvert = 0.928564254225894$	$\lvert H(q'') - H(q')\rvert = 1.200362176374867$
$\lvert H(r) - H(r')\rvert = 1.00827932372229$	$\lvert H(r'') - H(r')\rvert = 1.23278690031383$
$\lvert H(s) - H(s')\rvert = 1.004366651031468$	$\lvert H(s'') - H(s')\rvert = 1.043709139728033$
$\lvert H(t) - H(t')\rvert = 0.9941531277011$	$\lvert H(t'') - H(t')\rvert = 1.17381537511393$

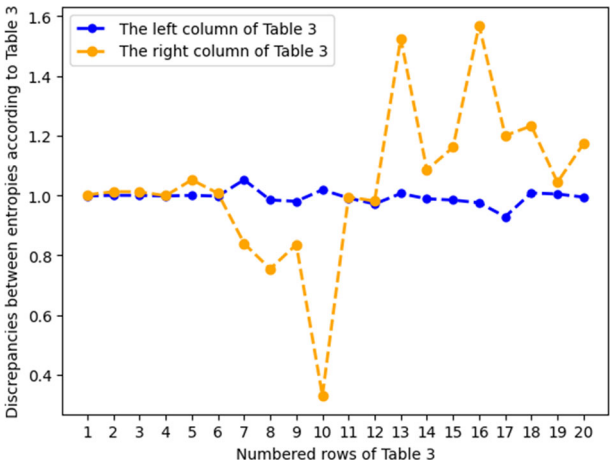


Fig. 7 The left and right columns of Table 3 represent the discrepancies of Shannon entropy of quantum circuits of an English sentence and its translation in Persian that shared the identical and non-identical angles in quantum circuits

circuits with different angles of rotation gates for synonymous words. The blue line exhibits a more regular pattern than the orange line. It is essential to acknowledge that noise and errors in quantum computing can affect the accuracy of these results, which is particularly significant for applications such as cryptography or chemical simulations that demand high precision and reliability. Figure 8 represents the heat map matrix representation of the pairwise entropy differences between all quantum circuits of sentences in Table 1.[2] The matrix represents differences between the first and second columns of Table 7. Entries on the main diagonal of the matrix are the values closest to unity. Figure 8 thus represents the absolute value of the pairwise entropy differences, minus unity. We have marked the location of these numbers in red dots as the best matches. By putting the same angles for rotation and controlled rotation gates for synonymous words in English and Persian sentences, according to Fig. 8, the distance between each English sentence and its translation in Persian is close to unity. Where the best matches are not on the diagonal, one can see by inspection that these cases arise from very similar syntactic structures between sentences (e.g. 'Sara walks in the park' vs. 'Bob walks in the park').

Noise can impact the outcomes obtained from quantum circuits executed on IBM quantum computers. Decoherence, the disturbance or loss of a qubit's quantum state due to interaction with the environment, is one of the major sources of noise in quantum computers. This can lead to errors in the quantum state and, consequently, impact the overall computation. Another source of noise is control errors, which arise during qubit manipulation, such as during gate operations, and can also impact the quantum state and the computation results. However, since noise and errors are inherent in quantum computing, it is essential to acknowledge their limitations. Therefore, the precision in our quantum approach is affected by this restriction and will be improved by future

[2] The numerical values for the 20×20 matrix elements representing the entropy differences between all quantum circuits in Table 1 are made available via an access link in Sect. 5.

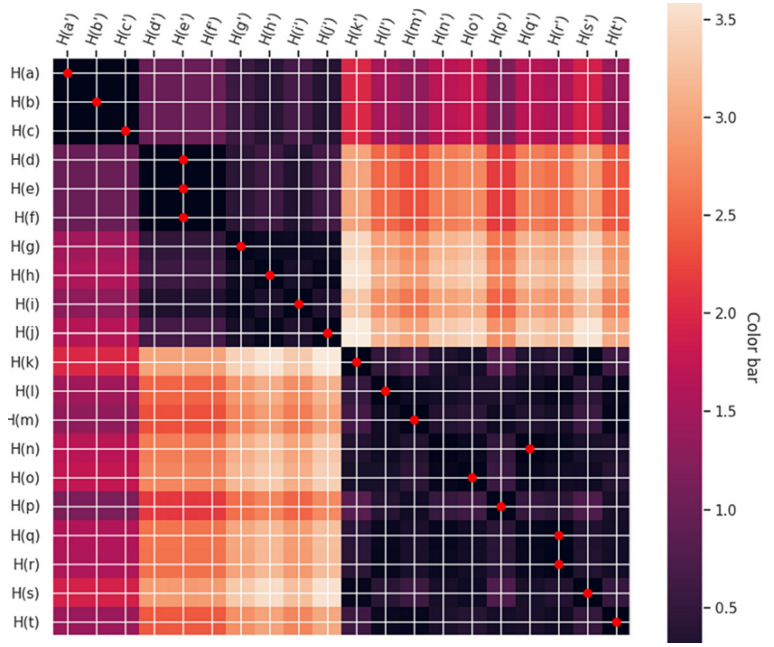


Fig. 8 The heat map representation of Shannon entropy differences between syntactically distinct languages (English and Persian) for finding the best translation for each sentence in a database. The heat map displays the absolute value of the pairwise entropy differences (from the left and middle columns of Table 2) minus unity. By subtracting these values from one, values near zero represent high translation similarity, while larger deviations indicate greater dissimilarity. Based on this representation, the entropy differences closest to unity are in the dark regions, mainly along the diagonal elements, shown by red dots on the heat map as the best match for translation. See main text for further details

generations of NISQ devices, which are expected to have significantly lower error rates.

4 Implementing the translation task via QNLP

Our test dataset in this section includes 160 sentences (augmented from the 20 sentences in the previous section), 80 sentences for each language in the translation task.[3] We considered the same angles of rotation gates for the synonymous words in quantum circuits of English and Persian sentences.

We have a classical computer interface, and the system works as a quantum–classical hybrid. The input to the machine translation network is derived from a decoded quantum circuit tailored to the network. Then, the result is converted back to a quantum circuit according to the encoding predicted by the network. Figure 9 shows the steps of the proposed method for encoding the quantum circuit into the LSTM input shape and then converting it back into the output quantum circuit in the target language.

[3] The access links to this dataset are available in Sect. 5.

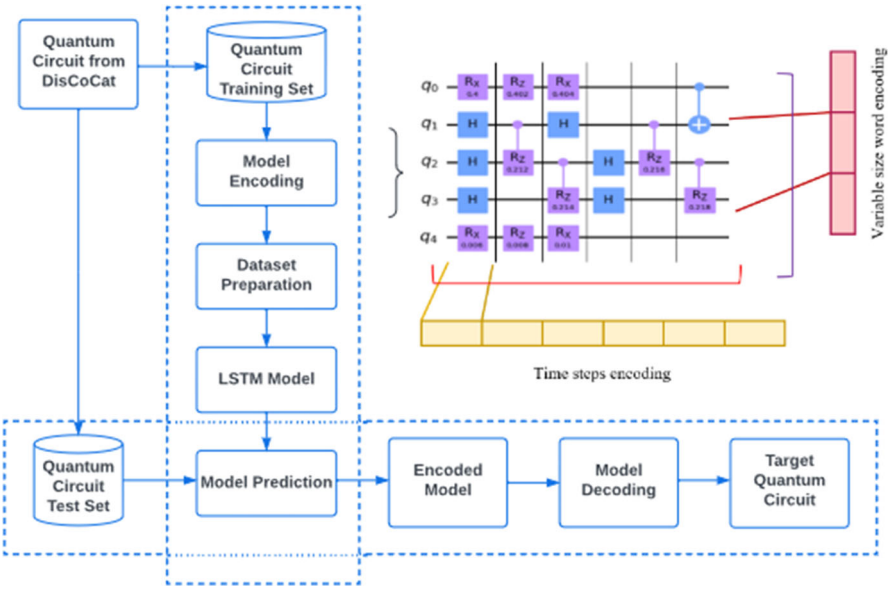


Fig. 9 Steps of the proposed method for the hybrid quantum–classical machine translation

4.1 Quantum-circuit encoding for translation task

To train our LSTM model using quantum circuits, we must first encode the circuits into a format that can be understood by the model and decode them back into the appropriate quantum circuit. This encoding should follow the same principles that feed sentences in a given language. However, this time, instead of encoding a sentence conventionally, the input is encoded from a quantum circuit. The result of encoding is represented as a nested vector of length $\mathbb{L}$ corresponding to the same number of words in the quantum circuit encoding. Our dataset D consists of source and target language sentence pairs, and we encode the set of quantum circuits QC from D. Each pair of quantum circuits $qcs, qct \in QC$ represents mappings for sentences in different languages using DisCoCat diagrams. For each quantum circuit in QC, its architecture is extracted and encoded as a vector E. The encoding E is constructed for the given DisCoCat-induced quantum circuit in a space corresponding to the quantum circuit model and is fed to the classical LSTM network. The network reads the quantum circuit from the first time step t_0 to the last time step t_{n-1} for every word in the sentence encoded from the quantum circuit. Each word has different time steps, each time step composed of a series of quantum gates, and each quantum gate has its own encoding. Therefore, we build a nested structure to prepare the encoding. Therefore, the encoding of each word w_l is constructed as a list of quantum gates in different time steps and is represented by $E_{w_l}(t^i_{w_l}(g_k))$.

$$E_{w_l} = [t^0_{w_l}, t^1_{w_l}, \cdots, t^{n-1}_{w_l}]$$

Here, E_{w_l} is a vector of time steps for word w_l in the quantum circuit. This vector has the same length for all words. Each element of this vector is represented as:

$$t^i_{w_l} = [[{g_1}^{w_l,i}], [{g_2}^{w_l,i}], \cdots, [{g_k}^{w_l,i}]]$$

$t^i_{w_l}$ shows gates inside word w_l for time step i. The left-hand side of the equation above is the symbol to represent the ith time step of the lth word in the sentence, and the right-hand side shows the vector of this word, which consists of k gates.

We represent each element in $t^i_{w_l}$ by a vector as above for $1 \leq j \leq k$, where k is the number of gates in time step i. k is not constant for each word, so we used padding to make them of equal size. $[{g_j}^{w_l,i}]$ is the vector representation for the gate g_j in time step i of word w_l. We used index j to refer to each gate in a given time step, and index i to refer to different time steps for the word.

Quantum circuits specify the evolution of the states in discrete time $t \in \mathbb{Z}$. Each time step corresponds to a unitary operation U_t, under which the state entering that time step evolves as $\mid \psi\rangle_t = U_t \mid \psi\rangle_{t-1}$. This time evolution is defined for words in different time steps. To separate the words in the quantum circuit, we use the fact that in DisCoCat, different words in a sentence do not have connections other than CNOT gates. The only connection that possibly exists between them is CNOT.

The main steps of the algorithm for processing and encoding the quantum circuit of a sentence are as follows:

- **Step 1:** Partitioning the quantum circuit into words.
- **Step 2:** Scheduling each partition in the quantum circuit into time steps.
- **Step 3:** Arranging gates in different time steps of each partition sequentially from top down.
- **Step 4:** Gate-level encoding of each quantum gate.

On the right side of Fig. 9 is a graphical representation of the encoding of a sample quantum circuit for the sentence "Sara sees Bob" with five qubits as an input to the LSTM network.

In this example, the state $|\psi\rangle = |q_0\rangle|q_1\rangle|q_2\rangle|q_3\rangle|q_4\rangle$ of the whole sentence is arranged into a vector of size equal to the number of words, and so after qubits are partitioned, we have: $P_{qc} = [|q_0\rangle, |q_1\rangle|q_2\rangle|q_3\rangle, |q_4\rangle]$. In addition, after scheduling, each vector is a 6-dimensional array, which is the number of time steps in the quantum circuit in this example.

Ignoring the CNOT and without optimising the circuit for the depth, we have seven distinguishable time steps in this example. Since CNOTs connect different words in the quantum circuit, their encoding is not involved in the word encoding. In this quantum circuit, there are three words, and for example, the vector structure of word embedding for the second word is $E_{w_2} = [t^1_{w_2}, t^2_{w_2}, \cdots, t^6_{w_2}]$.

Then, elements in each 6-dimensional array in the vector have variable sizes, which depend on the type of quantum gates in the particular time step. For example, for the third time step of the second word, it is $t^3_{w_2} = [[{g_1}^{w_2,3}], [{g_2}^{w_2,3}]]$.

This encoding makes the decoding part of the network straightforward. LSTM delivers the output in the same format as the encoded quantum circuit.

By processing the input quantum circuit as above, we calculated the encoding of each time evolution for different words in the sentence separately. We applied zero-padding to the unitary operations of words with fewer qubits to ensure fixed-size inputs to the network.

Table 4 Per-time-step schedule by word (ignore CNOT; keep CRZ; measurements excluded)

Time step	Sara (q_0)	Sees (q_1, q_2, q_3)	Bob (q_4)
t_1	Rx(q_0)	H(q_1), H(q_2), H(q_3)	Rx(q_4)
t_2	Rz(q_0)	$CRZ(q_1 \to q_2)$	Rz(q_4)
t_3	Rx(q_0)	H(q_1), $CRz(q_2 \to q_3)$	Rx(q_4)
t_4	–	H(q_2), H(q_3)	–
t_5	–	$CRz(q_1 \to q_2)$	–
t_6	$H(q_0)$	$CRz(q_2 \to q_3)$	–
t_7	–	H(q_3)	–

Example: Step-by-step encoding of a 5-qubit sentence circuit

To make the encoding procedure concrete, we illustrate it using the quantum circuit of the sentence *"Sara sees Bob"* that is presented in the dataset. The sentence is modelled as a 5-qubit circuit composed of three lexical components: subject, verb, and object (Table 4).

Step 1—Partitioning. The full sentence circuit acts on five qubits:

$|\psi\rangle = |q_0\rangle|q_1\rangle|q_2\rangle|q_3\rangle|q_4\rangle$

Following the grammatical structure of the DisCoCat model, the qubits are partitioned into word-level subcircuits as $P_{qc} = [|q_0\rangle, |q_1\rangle|q_2\rangle|q_3\rangle, |q_4\rangle]$ Here, q_0 represents the subject (*Sara*), qubits q_1–q_3 form the verb (*sees*), and q_4 represents the object (*Bob*).

Step 2, Step 3—Scheduling into time steps and arranging gates. Each word in the sentence is scheduled into 7 time steps:

$P_{qc}[0] = E_{w_1} = \left[t^1_{w1}, t^2_{w_1}, t^3_{w_1}, -, -, t^6_{w_1}, -\right];$

$P_{qc}[1] = E_{w_2} = \left[t^1_{w2}, t^2_{w_2}, t^3_{w_2}, t^4_{w_2}, t^5_{w_2}, t^6_{w_2}, t^7_{w_2}\right];$

$P_{qc}[2] = E_{w_3} = \left[t^1_{w3}, t^2_{w_3}, t^3_{w_3}, -, -, -, -\right]$

Each time step contains different gates for each word. To encode them, they are arranged into separate vectors as follows:

First word:

$t^1_{w_1} = \left[[g^{w_1,1}_1]\right]$; $t^2_{w_1} = \left[[g^{w_1,2}_1]\right]$; $t^3_{w_1} = \left[[g^{w_1,3}_1]\right]$; $t^6_{w_1} = \left[[g^{w_1,6}_1]\right]$; where $g^{w_1,1}_1 = R_x$, $g^{w_1,2}_1 = R_z$, $g^{w_1,3}_1 = R_x$, $g^{w_1,6}_1 = H$.

Second word:

$t^1_{w_2} = \left[[g^{w_2,1}_1], [g^{w_2,1}_2], [g^{w_2,1}_3]\right]$; $t^2_{w_2} = \left[[g^{w_2,2}_1], -\right]$; $t^3_{w_2} = \left[[g^{w_2,3}_1], [g^{w_2,3}_2]\right]$; $t^4_{w_2} = \left[-, [g^{w_2,4}_1], [g^{w_2,4}_2]\right]$; $t^5_{w_2} = \left[[g^{w_2,5}_1], -\right]$; $t^6_{w_2} = \left[-, [g^{w_2,6}_1]\right]$; $t^7_{w_2} = \left[-, -, [g^{w_2,7}_1]\right]$;

where $g^{w_2,1}_1 = H$, $g^{w_2,1}_2 = H$, $g^{w_2,1}_3 = H$, $g^{w_2,2}_1 = CRz$, $g^{w_2,3}_1 = H$, $g^{w_2,3}_2 = CRz$, $g^{w_2,4}_1 = H$, $g^{w_2,4}_2 = H$, $g^{w_2,5}_1 = CRz$, $g^{w_2,6}_1 = CRz$, $g^{w_2,7}_1 = H$.

Third word:

$t^1_{w_3} = \left[[g^{w_3,1}_1]\right]$; $t^2_{w_3} = \left[[g^{w_3,2}_1]\right]$; $t^3_{w_3} = \left[[g^{w_3,3}_1]\right]$;

where $g^{w_3,1}_1 = R_x$, $g^{w_3,2}_1 = R_z$, $g^{w_3,3}_1 = R_x$.

Table 5 Gate encoding scheme

Gate	Encoded value	Meaning
H	1	Hadamard gate
Rx(θ)	$2 + \theta$	Rotation around X axis
Rz(θ)	$3 + \theta$	Rotation around Z axis
CRz(θ) control	4	Controlled-Rz (control qubit)
CRz(θ) target	$5 + \theta$	Controlled-Rz target
No gate / padding	0	Idle position

Step 4—Gate-level numerical encoding. Each gate $g_j^{w_l,i}$ is converted into a numeric value that encodes its type and rotation parameter θ Table 5.

For example, $R_z(0.016)$ is encoded as 3.016 and $R_x(0.004)$ as 2.004.

Step 5—Padding and vector formation. Each word subcircuit may contain a different number of active qubits and gates per time step. To obtain a uniform input shape for the LSTM, zero-padding is applied so that all time-step vectors have equal length. So each E_{w_l} yields a fixed-length vector, and concatenating the three word vectors forms the full sentence representation:

$$E_{\text{sentence}} = [E_{w_1}, E_{w_2}, E_{w_3}].$$

Step 6—Decoding. The LSTM output preserves the same structure; each numerical entry can be mapped back to its corresponding gate type and parameter, thereby reconstructing the quantum circuit of the translated sentence.

Algorithm 1 Circuit-to-Vector Encoding

```
Input: Quantum circuit C
Partition qubits of C into word subcircuits P_qc
for each word subcircuit do
    Schedule gates into T sequential time steps
    for each gate g do
        Encode(g) ← type_id + rotation parameter
    end for
    Pad missing gates with zeros
end for
Concatenate word encodings → vector E
Output: Encoded vector E
```

This example demonstrates concretely how the process of partitioning, scheduling, gate-level encoding, and padding transforms a structured quantum circuit into a fixed-size numerical vector suitable for the LSTM.

4.2 Training classical neural network for language translation via QNLP

This section describes the classical–quantum encoder–decoder model for the translation task. The encoder reads the sentence to be translated, while the decoder translates it into the target language. Our encoding scheme transforms sentences into parameterised quantum circuits (PQCs), with each circuit having two layers. We ensure that synonymous words in different languages have the same angles for their rotation gates, although the circuit structures for different languages may differ. We trained a neural machine translation (NMT) model on the encoded representations of PQCs. The quantum circuit model is at the word level, as is the NMT model. We encode the features of the quantum circuit for each sentence and feed it as an input vector to a conventional long short-term memory (LSTM) encoder–decoder model. LSTM is a popular choice for *seq2seq* translation due to its ability to alleviate vanishing and exploding gradients, making it more effective than the original RNN model for sequence-to-sequence tasks such as machine translation. Finally, the output of the LSTM network is decoded back into the quantum circuit model of the sentence, with the corresponding encoded output of the network.

We utilised the pad_sequences function to ensure that all encoded sequences have the same length. Then, we constructed an encoder–decoder LSTM model for our task. In this model, each word in the sentence corresponding to the encoded sub-circuit in the quantum circuit is treated as a time step in a sequence.

In the first configuration, both the encoder and decoder consist of LSTM layers with 32 units. To connect these layers, we employed a RepeatVector layer, and finally, we wrapped a dense layer in a time-distributed wrapper. In the second configuration, we used two LSTM layers, each with 100 units, and a dense layer with 24 neurons, which corresponds to the dimension of each word in the quantum circuit.

Afterwards, we compiled the model using the basic stochastic gradient descent (SGD), RMSprop, and Adam optimisers and sparse categorical cross-entropy loss and trained it on the padded sequences. SGD is an optimisation algorithm that aims to minimise a function by employing a fixed learning rate throughout the training process. However, due to its tendency to exhibit high oscillation, increasing the learning rate becomes challenging, resulting in slower convergence. To address this issue, alternative optimisers have been developed. One such optimiser is RMSprop, which tackles the problem by adjusting the learning rate for each parameter individually. It accomplishes this by maintaining an exponentially decaying average of past squared gradients for every parameter. Similarly, the Adam optimiser also adjusts the learning rate for each parameter individually, utilising the history of gradients. This adaptive approach enables the optimiser to automatically scale the learning rate for each parameter, considering the specific characteristics of the observed gradients during training. In the context of LSTM networks, where distinct parameters may have varying ranges and properties (e.g. weights and biases of the input and forget gates), employing adaptive learning rates can lead to faster convergence and improved overall performance.

Our dataset consisted of 160 sentences in two languages: English and Persian. The LSTM model for the translation task is illustrated in Fig. 10.

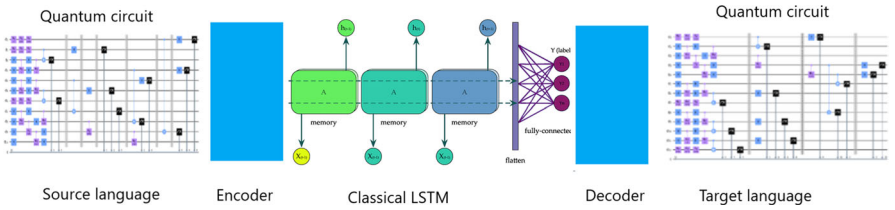


Fig. 10 Architecture of long short-term memory (LSTM) model training

4.3 Experimental results of implementing the translation task for QNLP

We utilised the dataset generated in the preceding section, consisting of 160 sentences, for training our encoder–decoder model. In our implementation, we employed the softmax activation function, a widely utilised choice for the output layer of LSTM-based machine translation models. This activation function serves the purpose of transforming the model's logits or scores into a probability distribution across the target vectors. The output layer of our model employed softmax as the activation function, while we used the Adam and RMSprop optimisers and sparse categorical cross-entropy loss during the training process. Our evaluation of the encoded quantum circuit dataset, consisting of English–Persian sentences, was performed using the trained network.

Table 6 presents the results of various models for quantum machine translation. We trained several LSTM models and report the performance of the top three models. The first model consists of two LSTM layers in the encoder: one LSTM layer in the decoder and a dense layer with a softmax activation function. The second model is a basic LSTM model with two LSTM layers and a softmax activation function. The last model combines an LSTM layer with RepeatVector, another LSTM layer, and a dense decoder layer in the output. In this context, the RepeatVector operation serves as a crucial bridge, facilitating the provision of the context vector to the second LSTM layer, which functions as the decoder. Through this mechanism, the RepeatVector operation plays a pivotal role in duplicating and propagating the encoded information from the encoder to the decoder, ensuring its availability for subsequent processing. In our experiment, the last model performs the best with the lowest mean absolute and squared errors.

Although our dataset is relatively small for a real-world machine translation task, we aim to showcase the combined architecture of a classical machine translation model with a quantum computer. Table 7 reports the validation loss for different epoch numbers using SGD, Adam, and RMSprop optimisers for 1000 epochs. The convergence of the validation loss over 1000 epochs for the Adam optimiser is shown in Fig. 11. The plot indicates that the loss function converges after approximately 200 epochs.

We re-implemented the LSTM translation setup, replacing the fixed quantum circuit encodings with fixed (frozen) pre-trained text embeddings. We used GloVe and fastText for source/target words as input representations. We kept the rest of the pipeline comparable: the same LSTM encoder→RepeatVector→LSTM decoder architecture (Model 3), evaluated MAE/MSE as auxiliary metrics, ran the three optimisers (SGD,

Table 6 Experimental results of each LSTM model for the translation task.

Model	MAE	MSE
Model 1	0.8156	2.8414
Model 2	0.6428	0.0021
Model 3	0.0324	0.0019

MAE = mean absolute error; MSE = mean squared error

Table 7 Experimental results of LSTM Model 3—with LSTM layer, RepeatVector, another LSTM layer, and dense decoder layer—for different optimizers

Optimizer	Loss
SGD	0.6423
Adam	0.01574
RMSprop	0.03078

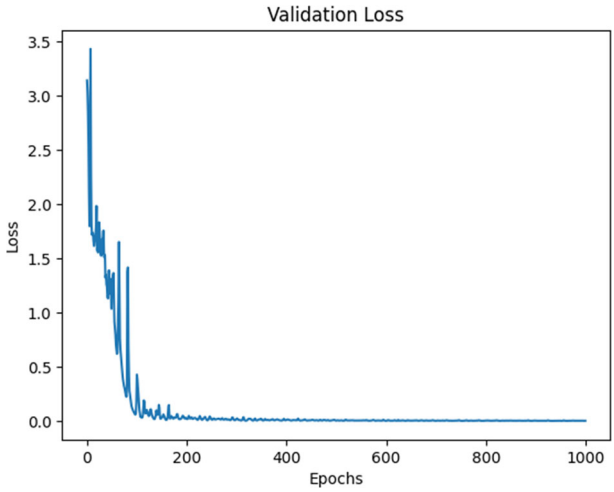


Fig. 11 Validation loss over 1000 epochs for LSTM Model 3 with Adam optimiser

Table 8 Evaluation metrics over 1000 epochs for LSTM Model 3 with different optimisers. The input encoding utilises GloVe for English sentences and fastText for Persian, with embeddings frozen (i.e. not updated during training)

Optimizer	MAE	MSE
SGD	0.128479	0.029722
Adam	0.182062	0.065136
RMSprop	0.161446	0.074457

RMSprop, Adam), and trained for 1000 epochs, with different random seeds and learning rates. This allows us to compare how different fixed input encodings (circuit vectors vs. GloVe/fastText) affect performance under the same training regimen; the results are shown in Table 8. The results indicate that the quantum encoding achieves a lower error rate for Model 3.

4.4 A short discussion on complexity

Quantum natural language processing (QNLP) may, in principle, offer advantages over classical methods in certain subroutines (e.g. compact encodings and black-box search) [48, 49]. Below, we quantify the costs of our workflow in both parts of the paper: the quantum-circuit realisation and the classical LSTM encoder–decoder.
(I) Quantum-circuit side (sentence circuits). Let Q be the number of qubits used for a sentence, T the scheduled time steps we keep for encoding (e.g. $T=7$ in our example; measurements excluded; CNOT ignored; CRZ kept), and let G_1 and G_{crz} be the numbers of single-qubit gates and controlled-R_z gates, respectively. The circuit *depth* is T, and the total gate count is $G_1 + G_{\mathrm{crz}}$ (since we ignore CNOT by design). Hardware execution time scales with T (latency) and gate durations; our encoding uses these same T steps as the temporal grid and does not change circuit asymptotics.
(II) Classical encode/decode + LSTM side Let L be the number of words in a sentence and K the per-time-step padding width (maximum gates retained for any word at any step after ignoring CNOT and excluding measurements). Our encoder scans the circuit once and writes at most $L \times T \times K$ scalars, so time and memory are $O(LTK)$; decoding has the same order. The LSTM consumes a tensor of shape (L, D) with $D=TK$, and hence, a forward pass scales as $O(LDH)$ for hidden size H, which dominates the lightweight $O(LTK)$ encode/decode overhead in practice. In our 5-qubit example ("Sara sees Bob"), $L=3$, $T=7$, and $K \leq 3$ for the verb ($K=1$ for nouns), so the per-sentence storage is only tens to a few hundred numbers.

5 Conclusion

In conclusion, this study aimed to investigate the potential for using quantum natural language processing algorithms on NISQ devices for language translation. By utilising Shannon entropy, we demonstrated the importance of angles in the performance of parametrised quantum circuits and showed how angles of rotation gates can be used as a means of communication between quantum circuits of different languages. We implemented the translation task using an encoder–decoder model with LSTM and achieved promising results on a small dataset of English and Persian sentences. However, we recognise that more complex machine translation models may be necessary to handle more intricate sentence structures. Our research lays the foundation for future work in quantum neural machine translation, which could offer advantages over classical methods in the future.

Acknowledgements This work was enabled by quantum computing access facilitated by the IBM-HBCU Quantum Centre at Howard University. We express our appreciation to Dr Sanjaya Lohani, Manny Gomez, and Dr Thomas Searles for helpful discussions on running quantum circuits and analysing the results.

Author Contributions M.A. has contributed to the main idea, dataset creation, evaluating Shannon entropy, mathematical proofs, and writing the draft of the manuscript. M.Z. has contributed to the main idea of the manuscript, writing the code, analysis of the results, and preparing the draft of the manuscript. V.S. contributed to the main idea of the manuscript, prepared the draft of the manuscript, analysed the results, and supervised the manuscript. M.S. supervised the project. P.K. contributed to the writing of the draft of the manuscript, revised the manuscript, analysed the results, and supervised the project.

Data Availability The authors declare that all data supporting the findings of this study are available within the article and its Supplementary Information files. Source data for implementing the translation task and 160 quantum circuits for English and Persian sentences are available at: https://github.com/Mina-Abbaszade/Data/blob/main/Translate(English).ipynb and https://colab.research.google.com/github/Mina-Abbaszade/Data1/blob/main/Translate(Persian).ipynb.
The numerical matrix representation of all differences between Shannon entropies of quantum circuits in Table 1 are available at: https://github.com/Mina-Abbaszade/Data/raw/main/Matrix_Representation.ipynb.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Turing, A.: Computing machinery and intelligence. Mind **236**, 433–460 (1950)
2. Popel, M., et al.: Interpretable quantum advantage in neural sequence learning, prx quantum 4. Nature Communications 020338 (2023)
3. Anschuetz, E.R., et al.: Transforming machine translation: a deep learning system reaches news translation quality comparable to human professionals. Nat. Commun. **11**, 4381 (2020)
4. Brown, T., et al.: Language models are few-shot learners. arXiv:2005.14165 (2020)
5. Fuchs, K.: Exploring the opportunities and challenges of NLP models in higher education: is chat GPT a blessing or a curse? Front. Educat. **8**, 1166682 (2023)
6. Lorenz, R., et al.: QNLP in practice: running compositional models of meaning on a quantum computer. J. Artif. Intell. Res. **76**, 1305–1342 (2023)
7. Du, S.L., Santana, S.H., Scarpa G.: A gentle introduction to Quantum Natural Language Processing. arXiv:2202.11766 (2022)
8. Wu, Y., et al.: Strong quantum computational advantage using a superconducting quantum processor. Phys. Rev. Lett. **127**(18), 180501 (2021)
9. Wu, Y., et al.: 40 years of quantum computing. Nat. Rev. Phys. **4**(1), 9–10 (2022)
10. Arute, F., et al.: Quantum supremacy using a programmable superconducting processor. Nature **574**(7779), 505–510 (2019)
11. Biamonte, J., et al.: Quantum machine learning. Nature **549**(7671), 195–202 (2017)
12. Schuld, M., et al.: An introduction to quantum machine learning. Contemp. Phys. **51**, 172–185 (2015)
13. Maria, S., Killoran, N.: Quantum machine learning in feature Hilbert spaces. Phys. Rev. Lett. **122**(4), 040504 (2019)
14. Maria, S., et al.: Circuit-centric quantum classifiers. Phys. Rev. A **101**(3), 032308 (2020)
15. Preskill, J.: Quantum computing in the NISQ era and beyond. Quantum **2**, 79 (2018)
16. Butler, K.T., et al.: Machine learning for molecular and materials science. Nature **559**(7715), 547–555 (2018)
17. Havlicek, V., et al.: Supervised learning with quantum-enhanced feature spaces. Nature **567**(7747), 209–212 (2019)
18. Liu, S.A.Y., Temme, K.: A rigorous and robust quantum speed-up in supervised machine learning. Nat. Phys. **17**(9), 1013–1017 (2021)
19. Huang, H.-Y., et al.: Quantum advantage in learning from experiments. Science **376**(6598), 1182–1186 (2022)
20. Cong Iris, S.C., Lukin, M.D.: Quantum convolutional neural networks. Nat. Phys. **15**(12), 1273–1278 (2019)
21. Daley, A.J., et al.: Practical quantum advantage in quantum simulation. Nature **607**(7920), 667–676 (2022)
22. Cerezo, M., et al.: Challenges and opportunities in quantum machine learning. Nat. Comput. Sci. **2**(9), 567–576 (2022)
23. Xiao, T., Zhai, X., Wu, X., Fan, J., Zeng, G.: Practical advantage of quantum machine learning in ghost imaging. Commun. Phys. **6**, 171 (2023). https://doi.org/10.1038/s42005-023-01268-0

24. Senokosov, A., Sedykh, A., Sagingalieva, A., Kyriacou, B., Melnikov, A.: Quantum machine learning for image classification. Mach. Learn.: Sci. Technol. **5**(1), 015040 (2024)
25. Xiao, T., Zhai, X., Huang, J., et al.: Quantum deep generative prior with programmable quantum circuits. Commun. Phys. **7**, 276 (2024). https://doi.org/10.1038/s42005-024-01765-9
26. Coecke, B., Felice, G., Meichanetzidis, K., Toumi, A.: Foundations for Near-Term Quantum Natural Language Processing. arXiv:2012.03755 (2020)
27. O'Riordan, L., Doyle, M., Baruffa, F., Kannan, V.: A hybrid classical-quantum workflow for natural language processing. Mach. Learn. Sci. Technol. **2**, 015011 (2020)
28. Coecke, B., Kissinger, A.: Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning. Polimetrica, Cambridge University Press, Cambridge (2017)
29. Abramsky, S., Coecke, B.: A categorical semantics of quantum protocols. In: 19th, Annual IEEE Symposium on Logic in Computer Science, 415–425 (2004)
30. Coecke, B., Sadrzadeh, M., Clark, S.: Mathematical foundations for a compositional model of meaning. Linguist. Anal. **36**, 345–384 (2010)
31. Sadrzadeh, M., Clark, S., Coecke, B.: The frobenius anatomy of word meaning i: subject and object relative pronouns. J. Log. Comput. **23**(6), 1293–1317 (2013)
32. Salehi, T., et al.: An optimizing method for performance and resource utilization in quantum machine learning circuits. Sci. Rep. **12 1**, 16949 (2022)
33. Coecke, B., Duncan, R.: Interacting quantum observables: categorical algebra and diagrammatics. New J. Phys. **13**(043016), 1394–1404 (2011)
34. Meichanetzidis, K., Gogioso, S., Felice, G.D., Chiappori, N., Toumi, A., Coecke, B.: Quantum natural language processing on near-term quantum computers. arXiv:2005.04147 (2020)
35. Kartsaklis, D., et al.: lambeq: An Efficient High-Level Python Library for Quantum NLP. arXiv:2110.04236, 2021 (2020)
36. Abbaszade, M., et al.: Application of quantum natural language processing for language translation. IEEE Access **9**, 130434–130448 (2021)
37. Monti, J.: Multi-Word Unit Processing in Machine Translation. PhD thesis, Universita Degli Studi Di Salerno (2013)
38. Nieto, I.V.: Towards Machine translation with Quantum Computers. Master's Degree Project, Physics. Stockholm, Sweden (2021)
39. Narayan, R., et al.: Quantum neural network based machine translator for english to hindi. Appl. Soft Comput. **38**, 1060–1075 (2016)
40. Schuld, M., et al.: The quest for a quantum neural network. Quantum Inf. Process. **13**, 2567–2086 (2014)
41. García, D.P., et al.: Systematic Literature Review: Quantum Machine Learning and its applications. arXiv:2201.04093 (2022)
42. Caro, M.C., et al.: Generalization in quantum machine learning from few training data. Nat. Commun. (2022)
43. Jung, H., et al.: Impact of sentence representation matching in neural machine translation. Appl. Sci. **12**(3), 1313 (2022)
44. Coecke, B.: Quantum picturalism. Contemp. Phys. **51**, 59–83 (2010)
45. Grefenstette, E., et al.: Concrete sentence spaces for compositional distributional models of meaning. In: Proceedings of the 9th International Conference on Computational Semantics (IWCS11) (2011)
46. Sadrzadeh, M.: Pregroup analysis of persian sentences. Computational algebraic approaches to natural language, In C. Casadio and J. Lambek, editors. Polimetrica (2006)
47. Lambek, J., C. Casadio, E.: Computational algebraic approaches to natural language. Polimetrica (2006)
48. W, Z., Coecke, B.: Quantum algorithms for compositional natural language processing. Polimetrica (2016)
49. Coecke, B., de Felice, G., Meichanetzidis, K., Toumi, A.: Foundations for near-term quantum natural language processing. Polimetrica (2020)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.